# Progettazione Modulo Didattico: Fondamenti AI/LLM, Git e RAG con Unsloth Studio e OpenCode

**Corso:** Tecnico Superiore per la Digitalizzazione dei Processi Industriali – ITS MAKER Bologna
**Destinatari:** 20–24 studenti tecnici (neofiti AI/LLM). Laboratorio a coppie con ruoli alternati (*Operatore* e *Collaudatore*).
**Hardware e Budget:** Postazioni con budget di risorse assegnate (4 vCPU, 8 GB RAM, no GPU dedicata) da misurare empiricamente su host VMware; assenza di garanzie a priori sulle prestazioni di picco condivise.
**Durata:** 6 ore base (360 min tot: 315 min attività nette, 45 min pause/margini; obiettivo ~80% attività nette al banco) / 8 ore estese (+120 min).

---

## 1. Obiettivi Didattici Osservabili e Concetti Chiave

- **Inferenza vs Addestramento:** lo studente spiega con un esperimento la differenza fra usare il modello e modificarne i pesi. La chat può conservare cronologie o memorie applicative senza addestrare il modello.
- **RAG (Retrieval-Augmented Generation):** inietta passaggi documentali pertinenti nella context window; non riaddestra il modello e non ne altera i pesi neurali.
- **Plausibilità vs Verità:** un modello linguistico genera testo sintatticamente coerente anche quando inventa dati (allucinazioni). L'output richiede costante validazione umana rispetto alle fonti primarie.
- **Ruoli Distinti:** Pesi LLM (modello statico), Runtime di inferenza (Unsloth Studio su base GGUF), Agente operativo (OpenCode con accesso a file locali via tool).
- **GGUF e Quantizzazione:** GGUF è un formato file contenitore; la quantizzazione riduce la precisione numerica dei pesi (es. FP16 a 4-bit o 2-bit). Un modello da 1 miliardo di parametri (1B) a 16-bit richiede 2 GB decimali di soli pesi; la RAM totale include runtime e memoria per la KV cache di contesto.
- **Nessun Dogma:** nessuna soglia magica a 7B, nessun vincitore prestabilito a priori e nessuna tabella tok/s teorica non misurata sul campo.

---

## 2. RAG Nativo in Unsloth Studio: Knowledge Base Locale

Il percorso usa la base documentale indicizzata associata alla chat e lo store SQLite di Studio. Non richiede un database esterno né presume la presenza, in ogni release, di un menu per creare una KB nominata: 1. **Configurazione Embedding:** impostato in `Settings > General > Documents & RAG`. Il default upstream è `bge-small-en-v1.5` (inglese); per il corpus tecnico italiano il docente pre-configura un modello di embedding multilingue verificato in cache locale. 2. **Associazione Documenti alla Chat:** all'interno della sessione chat, tramite l'icona `+ > Chat with Files > Add`

`files to chat with`, si associano i file del corpus. 3. **Persistenza per Chat:** ogni nuova chat richiede la riassociazione del corpus documentale; non si deve presumere un indice globale condiviso tra conversazioni indipendenti. 4. **Controllo Whole-Doc:** `laboratorio/avvia_studio.py` imposta nel processo `RAG_THREAD_WHOLE_DOC=0`, controllo presente nel sorgente upstream per evitare l'inserimento integrale dei piccoli allegati. Non è una garanzia inter-versione: il docente deve osservare il recupero dei passaggi e aprirne le fonti. 5. **Procedura Aggiornamento Documentale:** per testare la manutenzione documentale, si rimuove `03_catalogo_ricambi.md` (rev1: P03 -> FLT-992), si carica `03_catalogo_ricambi_rev2.md` (rev2: P03 -> FLT-993) e si apre una nuova chat riassociando il corpus aggiornato, verificando che la query T01 restituisca FLT-993 mentre lo storico guasto del 14/10/2026 mantenga traccia di FLT-992.

---

## 3. Matrice Query Canoniche T01–T06 e Valutazione Umana Separata

Gli studenti registrano ogni esito nel tracciato `risultati.csv` (UTF-8, separatore `;`). La valutazione umana separa rigorosamente l'esito del recupero dei passaggi dall'esito della risposta generata (`OK`, `ERRORE`, `NON_ESEGUITO`).

| Caso | Criterio osservabile | Fonti da verificare |
|---|---|---|
| T01 · Catalogo | Identifica filtro, macchina e revisione corrente, distinguendoli dalla registrazione storica. | DOC-RICAMBI |
| T02 · Calcolo | Recupera temperatura e soglia del rapporto; verifica il superamento con un calcolo indipendente. | DOC-FERMO |
| T03 · Quantità | Non confonde quantità ordinata, consegnata e mancante. | DOC-BOLLA |
| T04 · Parafrasi | Trova la durata del fermo anche se la domanda usa «indisponibilità». | DOC-FERMO |
| T05 · Dato assente | Riconosce l'assenza di evidenza; si astiene senza valori o citazioni inventati. | Nessuna fonte contiene il dato richiesto |
| T06 · Distrattore | Distingue le due macchine simili e cita entrambe le schede pertinenti. | DOC-P03 e DOC-P30 |

---

## 4. Architettura d'Aula a Tre Livelli e Limiti di Concorrenza

1. **Piano A – Locale su VM (Scenario Base):** Avvio tramite wrapper locale dalla root del repository:

   ```
   python laboratorio/avvia_studio.py --offline
   ```

   Lo script imposta nel processo figlio le modalità offline Hub/Transformers e il controllo whole-doc, senza disabilitare i tool RAG. Non è un firewall: disattivare web search e provider remoti e verificare l'avvio senza rete. Un rallentamento dell'host VMware può coinvolgere più VM.

2. **Piano B – LAN d'Aula per OpenCode (Scenario Docente):** Il docente espone un server dedicato per OpenCode su porta distinta (8889) per non interferire con la porta 8888 di Unsloth Studio:

```
unsloth run --model unsloth/Qwen2.5-Coder-7B-Instruct-GGUF:Q4_K_M -H 0.0.0.0 -p 8889 --disable-tools
```

   *Note tecniche:* `--disable-tools` è una scelta esplicita per lasciare gli strumenti a OpenCode. Configurare nel client il provider OpenAI-compatible con URL `http://IP_DOCENTE:8889/v1`, modello e chiave realmente generata in `Settings > API`; il comando locale non seleziona da solo questo server. Limitare la porta alla LAN e revocare le chiavi a fine laboratorio. La concorrenza sostenibile va misurata: non è garantita dal numero di parametri o dalla sola connettività.
3. **Piano C – Fallback Cloud (Emergenza):** Uso di endpoint cloud compatibili OpenAI (es. OpenRouter). Soggetto a limiti di traffico (free tier base: 20 RPM, 50 req/giorno) e vincolato all'uso **esclusivo di dati sintetici didattici**, senza invio di dati industriali riservati.

---

## 5. Strumenti Operativi: Git Essenziale e OpenCode

- **Configurazione Identità:** rigorosamente locale al repository dello studente (`git config user.name "..."` e `git config user.email "..."`), mai globale a livello di sistema.
- **Comandi Ammessi:**
- `git status`: ispezione file modificati e staging area.
- `git diff`: differenze tra working tree e indice di staging (`git diff HEAD` per confrontare col commit).
- `git add <file>` e `git commit -m "..."`: checkpoint atomici prima di ogni intervento.
- `git log --oneline`: visualizzazione cronologica sintetica.
- `git restore -- <file>`: ripristina modifiche non indicizzate (non staged) su file già tracciati. Non rimuove file untracked e non annulla commit storici. Vietati comandi distruttivi (`git clean -fd`, `git reset --hard`).
- **OpenCode e Report:** nella cartella studente (creata da `prepara_laboratorio.py`), lo studente esegue `python genera_report.py` (presente nella root del workspace) e guida OpenCode nell'aggiunta di un filtro client-side ("*Solo da rivedere*") per righe `ERRORE` o `NON_ESEGUITO`.
- **Sicurezza Didattica:** nessun collegamento a macchinari fisici. Lavorare solo nella cartella personale con permessi controllati. Chiedere al modello di trattare i documenti come dati è una mitigazione, non una protezione garantita dalle prompt injection o una sandbox del sistema operativo.

---

## 6. Checklist Golden Image e Matrice Go / No-Go

- **Script `preflight.py`:** registra Python, versioni CLI, opzioni help e impronte SHA256 del corpus; non collauda inferenza, RAG o carico. Un hash registrato permette un confronto successivo, ma non certifica da solo l'integrità.

- **Preparazione prima del corso:** scegliere e registrare release Studio/runtime, modello GGUF e revisione, quantizzazione, embedding multilingue, contesto e thread. Scaricare tutte le risorse prima dell'aula. Il riscontro locale della CLI 2026.10.6 non equivale a una release RAG qualificata.
- **GO soltanto dopo prove reali:** avvio senza rete, indicizzazione dei cinque file, recupero e citazioni osservati, esecuzione delle sei domande, aggiornamento rev2, report e creazione/modifica file tramite OpenCode. Misurare anche il carico delle 10–12 coppie sulla rete e sull'host reali.
- **NO-GO:** un backend alternativo è utilizzabile solo se già qualificato per lo stesso laboratorio. Un'API di generazione LAN/cloud non sostituisce da sola il recupero documentale. Risolvere i prerequisiti prima dell'aula; non sostituire le prove con risposte preparate.
- **Guida operativa:** laboratorio/docente/SOLUZIONI.md contiene preparazione, risposte attese e rubrica. Distribuire agli studenti solo lo starter; la separazione in cartelle nella repository pubblica non è una misura di segretezza.

---

## 7. Cronoprogramma Modulare (6 Ore Base / 8 Ore Estese)

| Blocco | Durata | Min Teoria | Min Pratica | Titolo Modulo e Attività Pratica |
|---|---|---|---|---|
| **B1** | 60 min | 10 | 50 | **Fondamenti AI/LLM:** esperimenti su conteggio token e contesto, distinzione modello/UI/agente. Studio è già avviato dal docente prima di B1. |
| **B2** | 30 min | 5 | 25 | **Git Essenziale:** setup locale, modifica `parametri_cnc.html` (3200 -> 4500 RPM), commit e prova mirata di ripristino. |
| **B3** | 30 min | 5 | 25 | **Unsloth e Configurazioni:** riuso di Studio, confronto di due configurazioni a impostazioni comparabili, misura RAM e latenza. Per isolare la quantizzazione deve restare uguale il modello. |
| **B4** | 90 min | 15 | 75 | **RAG Nativo in Studio:** associazione corpus officina in chat, esecuzione query T01–T06 su `risultati.csv`, aggiornamento a rev2 del catalogo e re-test. |
| **B5** | 45 min | 10 | 35 | **OpenCode e Report:** modifica a `genera_report.py`, filtro "Solo da rivedere", prova con fixture esplicitamente distinta dai risultati AI e commit. |
| **B6** | 60 min | 10 | 50 | **Collaudo Incrociato e Consegna:** inversione ruoli operatore/collaudatore, verifica incrociata dei CSV/report tra banchi, consegna repository. |
| *P* | 45 min | – | – | *Pause e margini distribuiti.* Totale 360 min: 315 min attività, di cui 250 di pratica prevista. Tempi da collaudare sull'aula, non prestazioni garantite. |
| **E1** | +60 min | 10 | 50 | **[Estensione 8h] Tuning RAG a Variabili Isolate:** variazione controllata di un parametro alla volta (chunk, overlap, prompt) su query in italiano. |
| **E2** | +60 min | 10 | 50 | **[Estensione 8h] Corpus Custom e Sfida:** redazione scheda tecnica d'officina, scambio tra banchi, formulazione query trappola e autovalutazione. |

---

## 8. Fonti Primarie e Documentazione Ufficiale

- Unsloth Studio Chat & UI

- Unsloth Changelog Ufficiale
- Unsloth Studio API
- Integrazione Unsloth & OpenCode
- Sorgente RAG Config Unsloth (`config.py`)
- Sorgente RAG Vector Store Unsloth (`store.py`)
- OpenRouter Pricing & Rate Limits